



PROPS: Learning Stack Patterns for ROP Detection on Legacy ARM-based Devices

József Sándor, Roland Nagy, and Levente Buttyán

CrySyS Lab
Department of Networked Systems and Services
Budapest University of Technology and Economics

Motivation

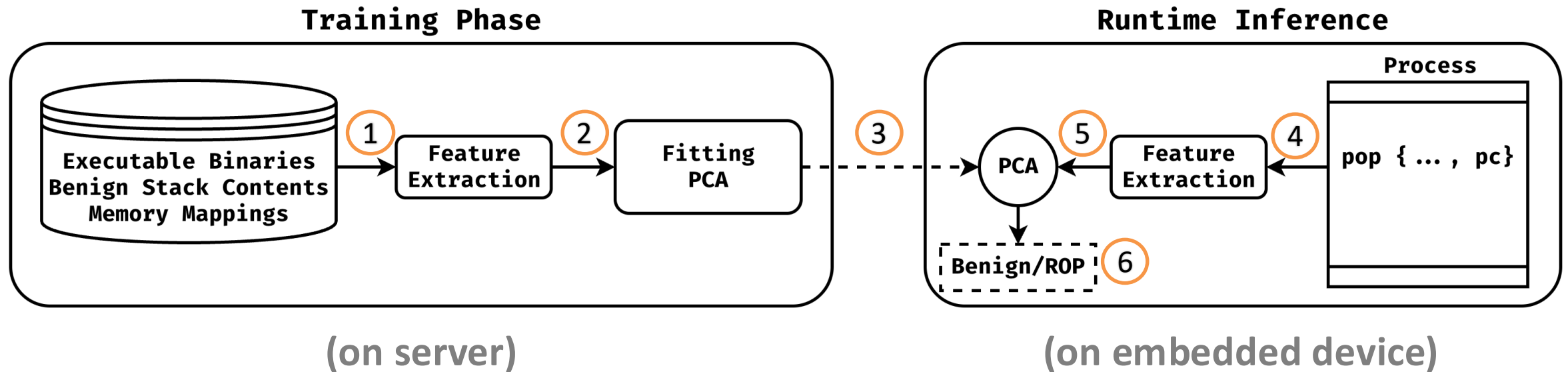
- Return-oriented programming (ROP) is still a viable attack
- No ultimate defense exists, especially for legacy embedded systems
- Modern hardware features on ARM architecture (PAC, BTI) are promising, but not available on devices with ARMv7 and older architectures
- In industrial and embedded systems
 - hardware upgrades are impractical
 - vulnerabilities cannot always be patched promptly
 - devices must continue to operate despite known memory-safety issues

Objectives

- For legacy systems with 32-bit ARMv7 and older architectures
- Easy to deploy and compatible with existing systems
- No modification of the existing software stack
- Detection vs Prevention
- Using the knowledge of a potential vulnerability (e.g., from CVE databases)
- Learn only the characteristics of benign stack content (anomaly detection)

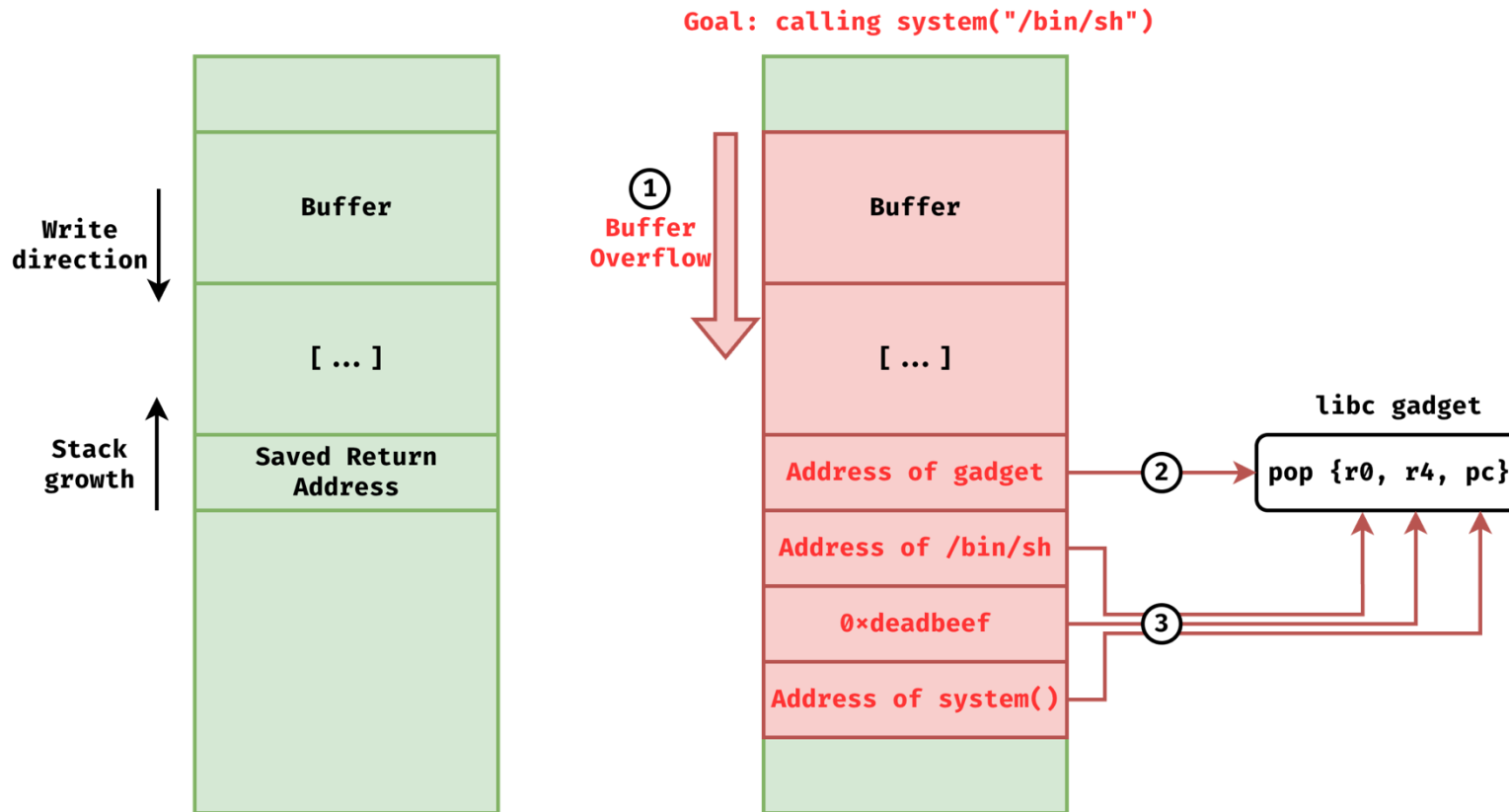
Overview of PROPS

- PROPS (**P**recision **ROP** Scanner)



ROP on 32-bit ARM

- Bypasses XN by chaining together small instruction sequences (gadgets)
- 32-bit ARM -> returning from function via `pop {..., pc}` instruction
- The chain is on the top of the stack



Feature Extraction Heuristic

- From the top of the stack, we take 32x4 bytes
- Every 4-byte value is considered as an address (4-byte aligned on 32-bit ARM)
- Check where it points, take 40 bytes (=10 instr.), and label it:
 1. **F** -> libc function
 2. **S** -> string
 3. **N** -> invalid instruction
 4. **!** -> SVC instruction (supervisor call)
 5. **G** -> pop {..., pc} instruction
 6. **J** -> branch-to-register instruction
 7. **X** -> executable region
 8. **W** -> writable region
 9. **R** -> readable region
 10. **?** -> address does not correspond to any mapped memory region

Example

benign stack state of /usr/bin/ls at pop {r4, r5, r6, pc}

W	?	?	X	?	W	W	?	X	W	W	X	W	W	?	W	?	?	W	N	X	?	N	G	?	?	?	W	?	R	W	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

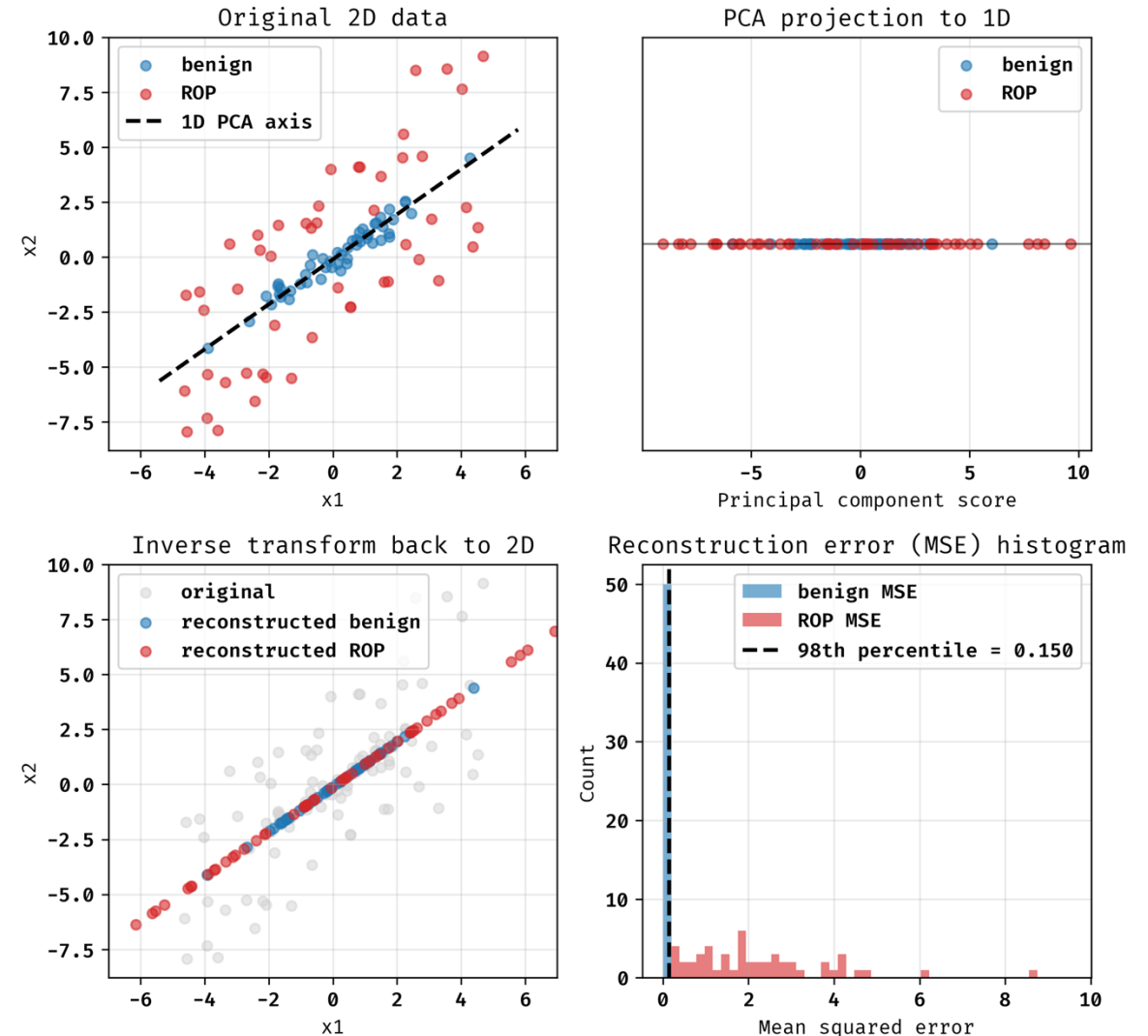
system("/bin/sh")

G	S	?	F
---	---	---	---

ROP as Anomaly on Stack

- Semi-supervised learning approach
- Using Principal Component Analysis (PCA)
- One-hot encoded feature vectors (32 x 10)
- Measuring reconstruction errors
 - low value -> normal, benign
 - high value -> anomaly, ROP
- Stack that contains ROP chain produce higher reconstruction error
- Threshold selection

PCA-based ROP Detection in 2D



Benign Stack Collection

- Raspberry Pi3 Model B running the 32-bit version of Raspberry Pi OS Lite
- Run 900+ programs on RPI3 with GDB
- Breakpoint at main()
 - Usually, every library is loaded at this point
 - We make a full memory dump
- Breakpoints at every pop {..., pc} instruction
 - save the top of the stack (32x4 bytes)
 - not every breakpoints are reached
 - same breakpoint may be hit multiple times
- 400K saved stack content (290K unique)
- 22K unique benign feature vectors



ROP Chain Generation

- Using angrop tool (Zeng et al., in NDSS 2026)
- 60 selected common ELFs (ls, grep, find, ...)
- 3 attacker objectives
 - Spawning a shell
 - Creating a new executable page
 - Making the stack executable
- 4 different methods by calling
 - functions using gadgets from libc
 - functions using gadgets from the target program
 - system calls using gadgets from libc
 - system calls using gadgets from the target program
- 2 modes: ARM and Thumb
- 1200+ ROP payloads (sometimes angrop didn't succeed)
- 284 unique feature strings after feature extraction

Example ROP Feature Vector

benign stack state of /usr/bin/ls at pop {r4, r5, r6, pc}

W	?	?	X	?	W	W	?	X	W	W	X	W	W	?	W	?	?	W	N	X	?	N	G	?	?	?	W	?	R	W	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

system("/bin/sh")

G	S	?	F
---	---	---	---

ROP chain on stack

W	?	?	G	S	?	F	?	X	W	W	X	W	W	?	W	?	?	W	N	X	?	N	G	?	?	?	W	?	R	W	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

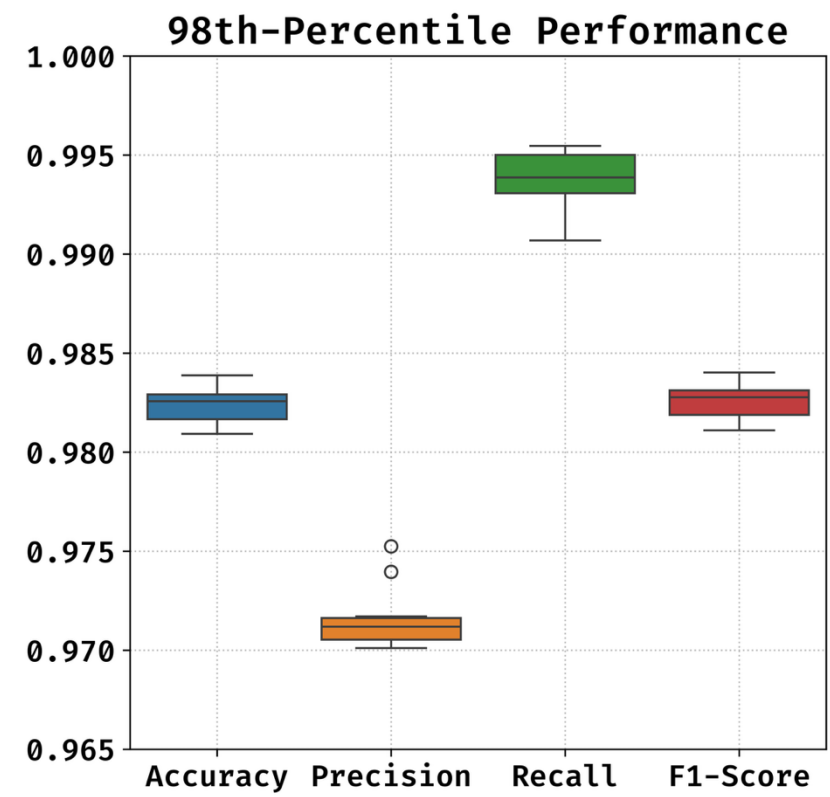
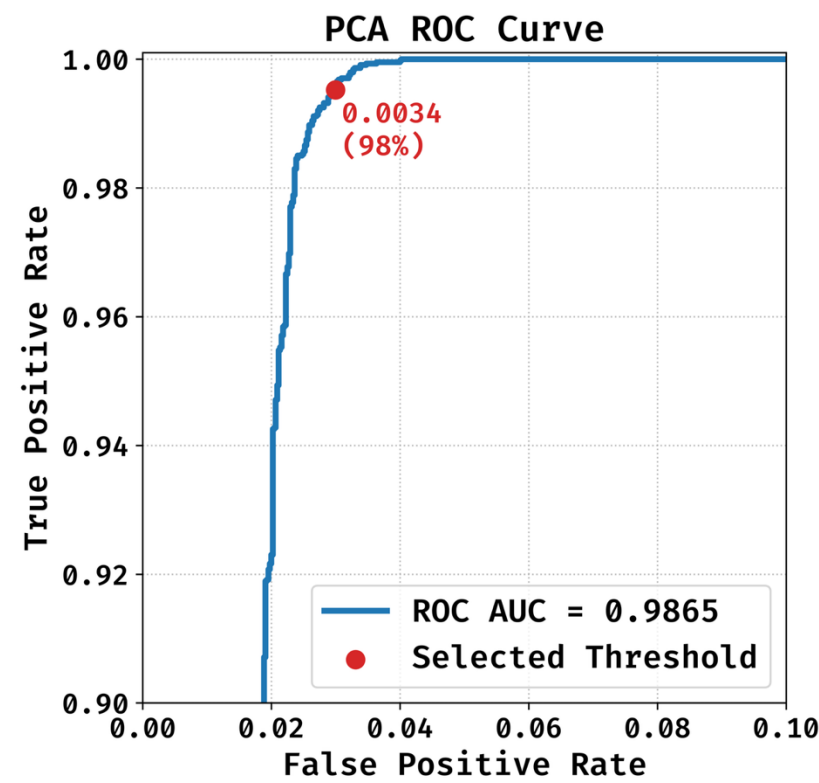

offset=3

Evaluation Setup

- Train on 80% of benign data (17, 600 samples)
- ROP chains are NOT used for training
- Test on 20% of benign data (4,400 samples)
- Insert ROP chains into benign data from test set (4,400 samples)
- Hyperparameter search (number of PCA components, threshold)
- K-fold cross validation (5 splits, 2 repeats)

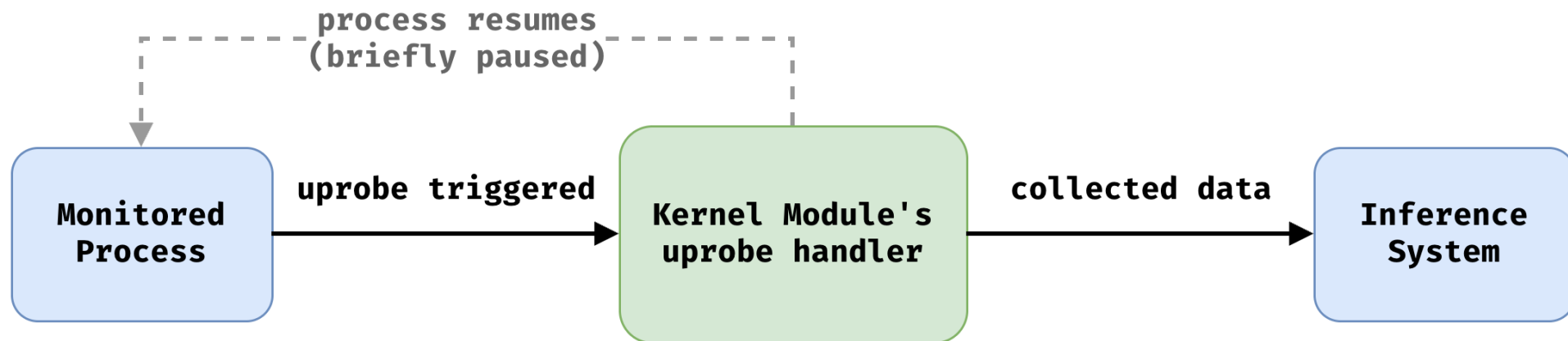
Detection Performance

- Used threshold: 98th percentile of reconstruction errors observed on the training set



Proof-of-Concept Implementation

- We place **uprobes** to some return instructions of the **monitored process** (to the “vulnerable function”)
- **Loadable Kernel Module** extracts the necessary data from a running process
- When the execution reaches an uprobe, the **Kernel Module** gets the control, extracts the data and sends to the **Inference System**
- **Monitored process** runs further



Runtime Performance

- Monitored process is paused while the kernel module collects the required data
- Kernel module processing time is fix except for the memory mapping traversal
- Low per-invocation latency
- Runtime overhead measured on 60 common utility programs
- A few trigger points placed at different function return points
 - measurements with 1, 3, 6 and 12 trigger points
 - same trigger point can be reached multiple times
 - 1-3 trigger points: +2% overhead on average
- Significant variation in individual program performance
 - depends on the target program and its arguments
 - measurement noise

Validation

- Crashmail 1.6 (CVE-2018-25223)
- PMS 0.42 (CVE-2018-25224)
- Using Exploit-DB ROP chains
- Successful detection

Conclusion

- PROPS is a ROP detection method for legacy ARM-based devices
- It is easy to deploy, compatible with existing systems
- Monitors only the return points of known (or potentially) vulnerable functions
- High detection rate (99% TPR)
- Practical runtime overhead with 1-3 trigger points
- Ongoing work
 - Validation and runtime measurements on additional vulnerable programs
 - Reducing FPR (2-3%) via ROP chain integrity check

Acknowledgment

- This work was supported by the National Research, Development and Innovation Fund of Hungary
 - Doctoral Excellence Fellowship Program
 - NKKP-ADVANCED_25-153325